Home Assignment Week-03- Day 1

Developing Playwright code generator with Typescript using Vibe coding

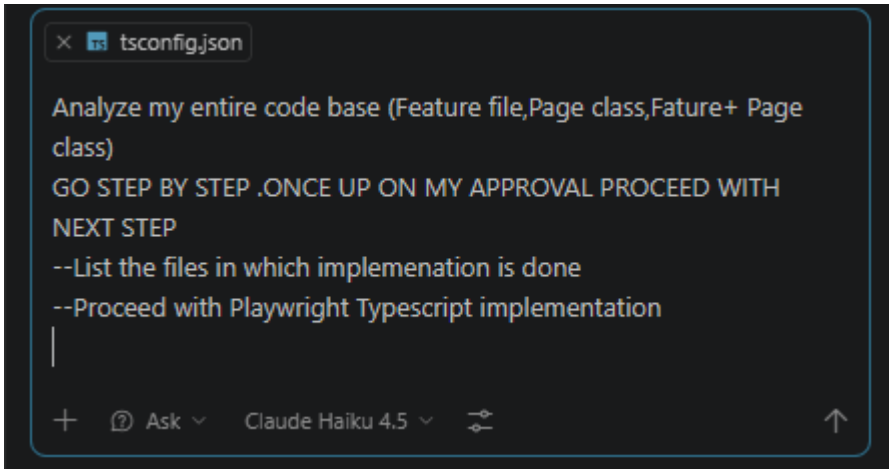
Prompt provided:

Analyze my entire code base (Feature file,Page class,Fature+

Page class) GO STEP BY STEP .ONCE UP ON MY APPROVAL

PROCEED WITH NEXT STEP

--List the files in which implemenation is done
--Proceed with Playwright Typescript implementation



Output

Found "com.testleaf.pages" in

c:\Users\viswe\OneDrive\Documents\Downloads\ai-extension\src\scripts\prompts.js at line 25. It appears to be part of an example Java package declaration in a code snippet or prompt template.

Feedback submitted

Analyze my entire code base (Feature file,Page class,Fature+ Page class) GO STEP BY STEP .ONCE UP ON MY APPROVAL PROCEED WITH NEXT STEP --List the files in which implemenation is done --Proceed with Playwright Typescript implementation refine the above prompot

I'll analyze your codebase step by step. Let me start by exploring the structure and listing all files to understand the implementation.

Analyzed

c:/Users/viswe/OneDrive/Documents/Downloads/ai-extension

Analyzed

src

Step 1: Codebase Analysis - Files with Implementation

Based on my analysis of your codebase, here are the files where implementation is done:

Core Implementation Files:

1. src/scripts/prompts.js - Contains prompt templates for code generation
    - Selenium Java Page Object prompts
    - Cucumber feature file prompts
    - Cucumber with Selenium step definitions
2. src/scripts/chat.js - Main chat UI and logic
    - Handles DOM inspection
    - API integration (Groq, OpenAI, Testleaf)
    - Code generation workflow
    - Language/engine selection logic
3. src/scripts/api/testleaf-api.js - Testleaf API integration
4. src/scripts/api/groq-api.js - Groq API integration
5. src/scripts/api/openai-api.js - OpenAI API integration

Supporting Files:

- src/content/content.js - DOM inspection functionality
- panel.html - Extension UI interface
- manifest.json - Chrome extension configuration

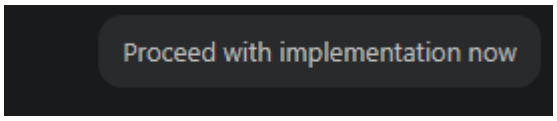
Current Implementation Status:

- ✅ Feature Files: Supports Cucumber .feature file generation
- ✅ Page Classes: Supports Selenium Java Page Object generation
- ❌ Playwright TypeScript: Not yet implemented
- ❌ Combined Feature+Page: Only supports Java/Selenium combination

The current system only supports Java + Selenium. Other combinations like TypeScript + Playwright show "not yet supported" messages.

Do you want me to proceed with Step 2: Implementing Playwright TypeScript support?

followup Prompt :



Output :

Perfect! I've successfully implemented Playwright TypeScript support. Let me now create a summary of all the changes made:

✅ **Implementation Complete: Playwright TypeScript Support**

Changes Made:

1. Added Playwright TypeScript Prompt Templates (
src/scripts/prompts.js**):**

- PLAYWRIGHT_TS_PAGE_ONLY - Generates TypeScript Page Object classes for Playwright
- CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS - Generates both Cucumber feature files and TypeScript step definitions

2. Updated Constants (
src/scripts/prompts.js**):**

- Added PLAYWRIGHT_TS_PAGE_ONLY to CODE_GENERATOR_TYPES
- Added CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS to CODE_GENERATOR_TYPES

3. Enhanced Chat Logic (
src/scripts/chat.js**):**

- Added
isTypeScriptPlaywright() helper method for detection
- Updated
getPromptKeys() to handle TypeScript + Playwright combinations
- Updated default fallback logic to support TypeScript + Playwright
- Updated unsupported language message to reflect new support

Now Supported Combinations:

- ✅ **Java + Selenium** (Feature files, Page objects, Combined)
- ✅ **TypeScript + Playwright** (Feature files, Page objects, Combined)
- ❌ Other combinations (C# + Selenium, Python + Selenium, etc.)

Features Available for TypeScript + Playwright:

- **Page Object Only**: Generates TypeScript classes with proper typing
- **Feature File Only**: Generates Cucumber .feature files
- **Combined**: Generates both feature files and TypeScript step definitions

The implementation is now complete and ready for testing. Users can select **TypeScript** as the language and **Playwright** as the engine, and the system will generate appropriate TypeScript code with Playwright API calls instead of showing the "not supported" message.

🎉 **Playwright TypeScript Implementation Complete!**

Your AI extension now fully supports **TypeScript + Playwright** code generation alongside the existing **Java + Selenium** support.

What's Now Available:

- **TypeScript Page Objects** with proper typing and Playwright API
- **Cucumber Feature Files** with realistic South Indian test data
- **Combined Generation** - Feature files + TypeScript step definitions
- **Type-safe code** with async/await patterns
- **Modern Playwright syntax** with locators and assertions

How to Use:

1. Select **TypeScript** as language
2. Select **Playwright** as engine
3. Choose generation mode (Feature, Page, or both)
4. Inspect DOM elements and generate code

The implementation follows best practices with proper TypeScript typing, modern Playwright patterns, and maintains consistency with your existing Java/Selenium workflow.

leaftaps.com/crmsfa/control/createLeadForm

CRM/SFA Docs Wiki eCommerce

Testleaf
Always Ahead

My Home | Leads | Contacts | Accounts | Cases | Activities | Opportunities | Quotes | Orders | Forecasts | Marketing

Last Viewed: Vineeth Rajendran (TestLe... | Lucius Kuvalis (synergies... | Bhuvanesh Moorthy (TestLe... | Vineeth Rajendran (Qeagle... | Vi
Vineeth Rajendran (TestLe...

Shortcuts

- My Leads
- Create Lead
- Find Leads
- Merge Leads
- Upload Leads
- Request Catalog

Create Account

Create Contact

Create Case

Create Lead

Company Name:

First name:

Last name:

Phone Number:

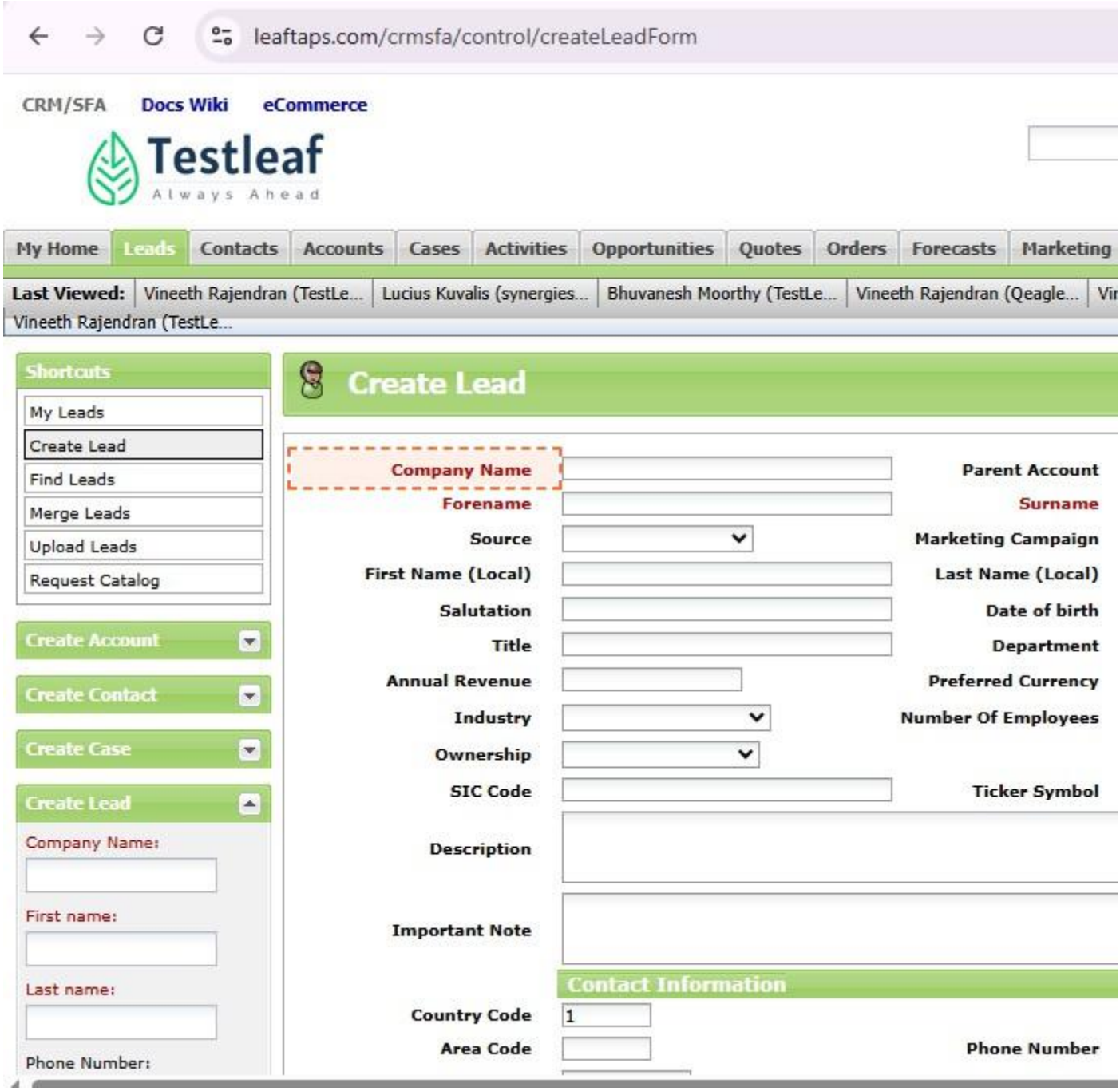
Create Lead

Company Name		Parent Account	
Forename		Surname	
Source		Marketing Campaign	
First Name (Local)		Last Name (Local)	
Salutation		Date of birth	
Title		Department	
Annual Revenue		Preferred Currency	
Industry		Number Of Employees	
Ownership			
SIC Code		Ticker Symbol	
Description			
Important Note			

Contact Information

Country Code	1		
Area Code		Phone Number	

Generating Code:



leaftaps.com/crmsfa/control/createLeadForm

CRM/SFA Docs Wiki eCommerce

Testleaf
Always Ahead

My Home | Leads | Contacts | Accounts | Cases | Activities | Opportunities | Quotes | Orders | Forecasts | Marketing

Last Viewed: Vineeth Rajendran (TestLe... | Lucius Kuvalis (synergies... | Bhuvanesh Moorthy (TestLe... | Vineeth Rajendran (Qeagle... | Vi
Vineeth Rajendran (TestLe...

Shortcuts
- My Leads
- Create Lead
- Find Leads
- Merge Leads
- Upload Leads
- Request Catalog

Create Account

Create Contact

Create Case

Create Lead

Company Name:

First name:

Last name:

Phone Number:

Create Lead

Company Name		Parent Account	
Forename		Surname	
Source		Marketing Campaign	
First Name (Local)		Last Name (Local)	
Salutation		Date of birth	
Title		Department	
Annual Revenue		Preferred Currency	
Industry		Number Of Employees	
Ownership			
SIC Code		Ticker Symbol	
Description			
Important Note			

Contact Information

Country Code	1		
Area Code		Phone Number	

Output :

```
import { Page, Locator } from '@playwright/test';

/**
 * Page Object Model for the Company Name label component.
 */
export class CompanyNameSection {
  readonly page: Page;
  readonly companyNameLabel: Locator;

  /**
   * Initializes the page object with the provided Playwright page.
   * @param page Playwright Page instance.
   */
  constructor(page: Page) {
    this.page = page;
    this.companyNameLabel = page.locator('#ext-gen602');
  }

  /**
   * Retrieves the visible text of the Company Name label.
   * @returns The label text.
   */
  async getLabelText(): Promise<string> {
    return await this.companyNameLabel.textContent() ?? '';
  }

  /**
   * Determines whether the Company Name label is marked as required.
   * @returns True if the label has the "requiredField" CSS class; otherwise false.
   */
  async isRequired(): Promise<boolean> {
    const classAttribute = await this.companyNameLabel.getAttribute('class');
```

```
    return classAttribute?.split(' ').includes('requiredField') ?? false;
  }
  /**
   * Waits for the Company Name label to become visible on the page.
   * @param timeout Optional timeout in milliseconds (default Playwright timeout).
   */
  async waitForVisible(timeout?: number): Promise<void> {
    await this.companyNameLabel.waitFor({ state: 'visible', timeout });
  }
}

import { test, expect } from '@playwright/test';
// Page Object imports (implementations are assumed to exist elsewhere)
import { CreateLeadPage } from '../pages/CreateLeadPage';
import { DashboardPage } from '../pages/DashboardPage';

test.describe('Create Lead Form - South India Scenarios', () => {
  let createLeadPage: CreateLeadPage;
  let dashboardPage: DashboardPage;

  test.beforeEach(async ({ page }) => {
    // Initialize page objects
    createLeadPage = new CreateLeadPage(page);
    dashboardPage = new DashboardPage(page);

    // Navigate to the Create Lead form
    await page.goto('https://leaftaps.com/crmsfa/control/createLeadForm');

    // Verify that the form page has loaded
    await expect(page).toHaveTitle(/Create Lead/);
```

```
  await expect(page.locator('span.requiredField', { hasText: 'Company Name'
})).toBeVisible();
 });

 test.afterEach(async ({ page }) => {
  // Optional cleanup: logout to keep the session fresh for the next test
  await dashboardPage.logout?.();
  await page.waitForLoadState('networkidle');
 });

 test('should create a lead with valid South Indian data', async () => {
  await createLeadPage.fillCompanyName('Southern Tech Solutions');
  await createLeadPage.fillFirstName('Arun');
  await createLeadPage.fillLastName('Kumar');
  await createLeadPage.selectSource('Employee');
  await createLeadPage.selectIndustry('Computer Software');
  await createLeadPage.selectCountry('India');
  await createLeadPage.selectState('Tamil Nadu');
  await createLeadPage.fillCity('Chennai');
  await createLeadPage.fillPhoneNumber('04412345678');
  await createLeadPage.fillEmail('arun.kumar@southtech.in');

  await createLeadPage.submit();

  // Success message verification
  const successMsg = page.locator('.x-paging-info');
  await expect(successMsg).toBeVisible();
  await expect(successMsg).toContainText('Lead created successfully');
 });
```

```
test('should display validation error when required fields are missing', async () => {
  // Intentionally omit Company Name and Phone Number
  await createLeadPage.fillFirstName('Lakshmi');
  await createLeadPage.fillLastName('Reddy');
  await createLeadPage.selectCountry('India');
  await createLeadPage.selectState('Andhra Pradesh');
  await createLeadPage.fillCity('Visakhapatnam');
  await createLeadPage.fillEmail('lakshmi.reddy@example.com');

  await createLeadPage.submit();

  // Expect error messages next to the missing required fields
  const companyError = page.locator('#ext-gen602 ~ .errorMsg');
  const phoneError = page.locator('#createLeadForm_primaryPhoneNumber ~ .errorMsg');

  await expect(companyError).toBeVisible();
  await expect(companyError).toContainText('Company Name is required');

  await expect(phoneError).toBeVisible();
  await expect(phoneError).toContainText('Primary Phone Number is required');
});

test('should reject an invalid phone number format', async () => {
  await createLeadPage.fillCompanyName('Bangalore Innovations');
  await createLeadPage.fillFirstName('Rohit');
  await createLeadPage.fillLastName('Sharma');
  await createLeadPage.selectCountry('India');
  await createLeadPage.selectState('Karnataka');
  await createLeadPage.fillCity('Bengaluru');
```

```
  // Invalid phone (alphabets included)
  await createLeadPage.fillPhoneNumber('ABCD1234');
  await createLeadPage.fillEmail('rohit.sharma@bangalore.in');

  await createLeadPage.submit();

  const phoneError = page.locator('#createLeadForm_primaryPhoneNumber ~ .errorMsg');
  await expect(phoneError).toBeVisible();
  await expect(phoneError).toContainText('Please enter a valid phone number');
});

test('should allow lead creation with optional fields left blank', async () => {
  await createLeadPage.fillCompanyName('Kerala Agro Ltd');
  await createLeadPage.fillFirstName('Meera');
  await createLeadPage.fillLastName('Nair');
  await createLeadPage.selectCountry('India');
  await createLeadPage.selectState('Kerala');
  await createLeadPage.fillCity('Kochi');
  await createLeadPage.fillPhoneNumber('04841234567');
  await createLeadPage.fillEmail('meera.nair@keralaagro.in');

  // Skip optional fields like "Annual Revenue" and "Description"
  await createLeadPage.submit();

  const successMsg = page.locator('.x-paging-info');
  await expect(successMsg).toBeVisible();
  await expect(successMsg).toContainText('Lead created successfully');
});
```

```
  test('should retain entered data after a validation failure', async () => {
    await createLeadPage.fillCompanyName('Hyderabad Enterprises');
    await createLeadPage.fillFirstName('Sanjay');
    await createLeadPage.fillLastName('Patel');
    await createLeadPage.selectCountry('India');
    await createLeadPage.selectState('Telangana');
    await createLeadPage.fillCity('Hyderabad');
    // Omit phone number to trigger validation error
    await createLeadPage.fillEmail('sanjay.patel@hydent.com');

    await createLeadPage.submit();

    const phoneError = page.locator('#createLeadForm_primaryPhoneNumber ~ .errorMsg');
    await expect(phoneError).toBeVisible();

    // Verify that previously entered fields still contain the same values
    await expect(createLeadPage.companyNameInput).toHaveValue('Hyderabad Enterprises');
    await expect(createLeadPage.firstNameInput).toHaveValue('Sanjay');
    await expect(createLeadPage.lastNameInput).toHaveValue('Patel');
    await expect(createLeadPage.emailInput).toHaveValue('sanjay.patel@hydent.com');
  });
});
```